

Laboratory Report

Search and A*: Warehouse Robot Navigation

Artificial Intelligence Laboratory | Using an LLM as an Engineering Assistant

Name / ID: _____

Summary

A* search with the Manhattan-distance heuristic was used to plan a path for a warehouse robot from S to G. The program found an optimal path of **40 moves**, expanding **64 states**, and passed all four required tests (original map, trivial case, no-solution case, alternative paths). BFS returned the same path with the same number of expansions. The investigation showed why: the lab warehouse has only 64 free cells, all of them reachable, and exactly one shortest path, so no informed search can prune anything on this map. Supplementary experiments on open rooms show where A* does help (169 states for BFS versus 15 for A*), and a small counterexample shows that the 2x Manhattan heuristic can return a non-optimal path (9 moves instead of 7).

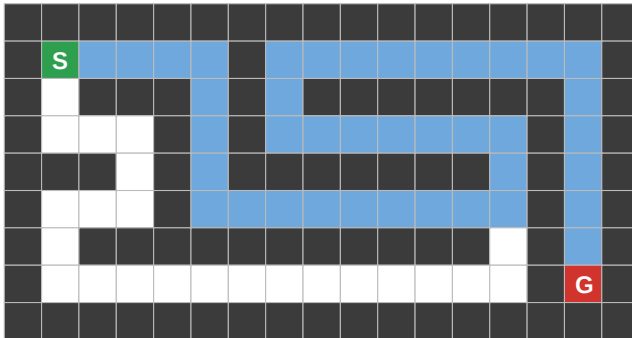


Figure 1. The warehouse with the A* path (blue), start S (green) and goal G (red).

1. Formulation of the Search Problem (Task 0)

The problem is written as $P = (S, A, T, s_0, G, c)$, as in the lecture.

Component	Specification
State S	A grid position (row, column) of the robot on a free cell. 64 free cells give 64 possible states.
Actions A	Up, Down, Left, Right (row/column changes $(-1,0)$, $(1,0)$, $(0,-1)$, $(0,1)$).
Transition T	$T((r,c), a) = (r+dr, c+dc)$ if that cell is inside the grid and is not '#'; otherwise the action is not available.
Initial state s_0	The cell marked S: (1,1).
Goal G	The cell marked G: (7,15). The goal test is $state == goal$.
Cost c	Every move costs 1, so path cost equals the number of moves.

(a) Information needed to specify a state. Only the robot's row and column. The map is fixed, so it is part of the problem, not of the state.

(b) Invalid action. One that leaves the grid or enters an obstacle '#'.

(c) Deterministic? Yes. A state and a valid action determine exactly one next state, and the environment is static and fully observable.

(d) Solution. A sequence of valid moves taking the robot from S to G. An *optimal* solution is one with minimum total cost.

2. Design of the Agent (Task 1)

Design question	Decision
State in Python	A tuple (row, col).
Warehouse	A list of strings (the ASCII map). <code>parse_grid</code> finds S and G.
Valid actions	<code>valid_neighbors(grid, state)</code> tries the four moves and keeps those inside the grid that are not '#'.
Goal recognition	Test <code>current == goal</code> when a state is removed from the frontier.
Frontier contents	A priority queue (binary heap) of tuples (f, g, counter, state). f orders the queue, g detects stale entries, and counter breaks ties so states are never compared.
Path reconstruction	A <code>came_from</code> dictionary maps each state to its parent. Follow it from the goal back to the start, then reverse.
Reported on termination	Whether a solution was found, the path, the path length, and the number of states expanded.

The evaluation function is $f(n) = g(n) + h(n)$, with $h(n) = |r - r_G| + |c - c_G|$ (Manhattan distance).

3. Implementation (Task 2)

The final program is `search_agents.py` (full listing in Appendix B). It contains A* with four interchangeable heuristics, a BFS version for comparison, path reconstruction, state-expansion counting, and the four required test maps. Only the Python standard library is used (`heapq`, `collections.deque`, `math`). The prompt used to obtain the A* implementation is given in Appendix A. Run it with `python search_agents.py`.

4. Test Results (Task 3)

Test 1: original warehouse. A path was found. Length 40, states expanded 64.

```
#####
#S****#*****#
#.#*#*#####*#
#..#*#*****#*#
###.#*#####*#*#
#..#*****#*#*#
#.#*#####.#*#
#.....#G#
#####
```

Path (row, col): (1,1) (1,2) (1,3) (1,4) (1,5) (2,5) (3,5) (4,5) (5,5) (5,6) ... (5,13) (4,13) (3,13) (3,12) ... (3,7) (2,7) (1,7) (1,8) ... (1,15) (2,15) (3,15) ... (7,15). The robot must go down the left side, across the bottom corridor, back up, across the top, and down the right side.

Test	Map	Found	Path length	States expanded	Expected / check
1 Original	lab map	True	40	64	Path valid; length matches BFS (40)
2 Trivial	#SG##	True	1	2	One-step solution
3 No solution	goal walled off	False	None	9	Terminates and reports failure; 9 = all reachable states

Test	Map	Found	Path length	States expanded	Expected / check
4 Alternatives	open 3x5 room	True	4	5	4 = Manhattan lower bound, so shortest

Notes on the tests. In Test 3 the program terminated after expanding exactly the 9 states reachable from S, so the closed set prevents infinite loops. In Test 4 the map contains many routes (detours through rows 2 and 3), but only the straight 4-move route is shortest, and A* returned it. The supplied test code only asserts that a path was found in Test 4, so the length was checked separately against BFS and the Manhattan lower bound.

5. Inspection of the A* Algorithm (Task 4)

Concept	Where it appears in <code>search_agents.py</code>
State	(row, col) tuples, e.g. <code>start</code> , <code>current</code> , <code>neighbor</code> (lines 136-210)
Action	The <code>MOVES</code> list (Up/Down/Left/Right), used in <code>valid_neighbors</code> (line 80)
Transition	<code>nr, nc = r + dr, c + dc</code> inside <code>valid_neighbors</code> ; result is yielded as a neighbour state
Goal test	<code>if current == goal</code> (line 179), applied after popping from the frontier
$g(n)$	<code>g_cost</code> dictionary and <code>tentative_g = current_g + 1</code> (lines 160, 190-193)
$h(n)$	<code>h = heuristic(neighbor, goal)</code> (line 197); functions <code>manhattan</code> , <code>euclidean</code> , etc. (lines 109-124)
$f(n)$	<code>new_f = tentative_g + h</code> (line 198), pushed as the first element of the heap tuple
Frontier	<code>frontier</code> , a heap managed with <code>heapq.heappush</code> / <code>heappop</code> (lines 150-170, 200)
Visited states	<code>expanded_g</code> (states already expanded, with their <code>g</code>) plus the stale-entry check against <code>g_cost</code> (lines 164-177)
Path reconstruction	<code>came_from</code> dictionary and <code>reconstruct_path</code> (lines 92-106)

(a) Frontier data structure. A binary heap (Python list used with `heapq`) of tuples (`f`, `g`, `counter`, `state`).

(b) Choosing the next state. `heappop` returns the tuple with the smallest `f`. Ties are broken by smaller `g`, then by insertion order (`counter`).

(c) Heuristic calculation. Once for the start state when the frontier is created, and once for every neighbour that is pushed (line 197).

(d) Explicit $f = g + h$? Yes: `new_f = tentative_g + h`.

(e) Preventing repeated exploration. Three mechanisms. A neighbour is pushed only if it gives a strictly cheaper `g` than any known route. Outdated heap entries are skipped when popped (`current_g != g_cost[current]`). A state already expanded at an equal or lower cost is skipped. Unlike a plain visited set, this still allows a state to be re-expanded if a cheaper route is found later, which matters for inadmissible heuristics (Section 7).

6. Comparison of A* with Blind Search (Task 5)

Measure	BFS	A* (Manhattan)
Solution found	True	True
Path length	40	40
States expanded	64	64

(a) Both found a solution. (b) Both found paths of length 40, which is optimal: BFS is optimal for unit costs, and Manhattan distance is admissible. (c) Neither expanded fewer states: both expanded 64.

(d) **Why A* can expand fewer states, and why it did not here.** A* uses h to favour states that look close to the goal. It expands every state with $f < C^*$ (the optimal cost, 40) before the goal, and never expands a state with $f > C^*$. On this map the start is at Manhattan distance 20 from the goal, but the real cost is 40, because the walls force the robot to travel away from the goal and back. The heuristic is therefore very optimistic, and every one of the 64 free cells has $f \leq 40$, so A* must visit all of them. The map is also essentially a single winding corridor with dead-end branches and only one shortest path, so there is nothing for a heuristic to prune.

Supplementary experiment: where A* does help

To check that the equal result is a property of this map and not a bug, both algorithms were run on open 15x15 rooms (code in Appendix C).

Map	Path length	BFS expanded	A* expanded
Lab warehouse (17x9 maze)	40	64	64
Open 15x15 room, S and G in the same row	14	169	15
Open 15x15 room, S and G in opposite corners	28	225	225
Same corner-to-corner room, A* tie-break on larger g	28	225	29

When S and G share a row, the Manhattan heuristic is exact along the row, so A* walks almost straight to the goal. In the corner-to-corner room, every cell on an optimal path has the same f , so the result depends on tie-breaking. The program's heap orders ties by *smaller* g , which behaves like BFS and expands everything; preferring *larger* g (deeper states first) focuses the search and cuts the work from 225 to 29 states. Informed search helps only when the heuristic discriminates between states, and details such as tie-breaking matter.

7. Heuristic Investigation (Task 6)

Why Manhattan distance suits this problem. Each move changes the row or the column by exactly 1, so it reduces $|dr| + |dc|$ by at most 1. Reaching the goal therefore needs at least $|dr| + |dc|$ moves, so $h(n) \leq h^*(n)$ (admissible). It is also consistent, so A* never needs to reopen a state.

Heuristic	Found	Path length	States expanded
Manhattan	True	40	64
Zero, $h(n) = 0$	True	40	64
Euclidean	True	40	64
2 x Manhattan	True	40	67

$h(n) = 0$. A* reduces to uniform-cost search (Dijkstra), which for unit costs explores like BFS. It is still optimal, but has no guidance. Here it expanded the same 64 states.

Euclidean. Admissible, because a straight line is never longer than a grid path, but weaker than Manhattan: $\text{Euclidean} \leq \text{Manhattan}$ everywhere, so Manhattan *dominates* it and never expands more states in principle. On this map there is no difference (64).

2 x Manhattan. Inadmissible: it over-estimates whenever the true remaining cost is less than twice the Manhattan distance, for example in any open area. On the lab map it still returned the optimal 40 moves, but needed 67 expansions, more than 64, because some states were expanded again after a cheaper route was found. An optimal answer here does not prove the heuristic is safe. A small counterexample (found by random search over 6x7 maps) shows the failure:

```
#####
#S...#
#.#.#.#
#.#...#
#...#G#
#####
```

Heuristic	Path length	States expanded
Manhattan	7 (optimal)	15
Zero	7 (optimal)	16
Euclidean	7 (optimal)	15
2 x Manhattan	9 (not optimal)	10

The 2x heuristic was overconfident about the direct-looking route down the left side and committed to a 9-move path, finishing with fewer expansions but a worse answer. **Conclusion:** an over-optimistic (too aggressive) heuristic makes A* greedier: it is faster but may lose optimality. An under-informed heuristic (zero) keeps optimality but loses speed. Admissibility guarantees optimality, and a heuristic closer to h^* prunes more.

8. Evaluation of the LLM-Generated Agent (Task 7)

Edit the first-person statements in this section so they match your own workflow and prompts.

- 1. Correct immediately.** The problem formulation, neighbour generation, path reconstruction, lazy deletion of stale heap entries, termination on unreachable goals, and the path lengths. BFS and A* agree on every test map.
- 2. Bugs or design problems.** No correctness bug was found. Design points: the tie-breaking rule prefers smaller g , which makes A* behave like BFS on open maps; the reported expansion count includes the goal itself; and the Test 4 assertion only checks that a path exists, not that it is shortest.
- 3. How they were discovered.** By cross-checking against BFS, by comparing the path length with the Manhattan lower bound, by running the heuristics on open rooms, and by randomly searching for a map on which 2x Manhattan fails.
- 4. Unfamiliar terminology.** The `counter` in the heap tuple (a tie-breaker that stops Python comparing states), `came_from` (parent pointers), and stale entries (heapq has no decrease-key, so duplicates are pushed and outdated ones skipped when popped).
- 5. Modified?** The algorithm was not changed. Supplementary experiments (open rooms, tie-breaking variant, counterexample search) were written separately so the original results stay reproducible.
- 6. Most useful tests.** The no-solution test (termination), the BFS cross-check (an independent oracle for optimal length), and the 2x Manhattan counterexample.
- 7. Could the program be trusted untested?** No. The Test 1 output looks plausible, but plausibility does not show optimality, and the equal BFS/A* counts would have hidden the fact that the heuristic was doing no work on this map.
- 8. What was learned about A*.** A* expands everything with $f < C^*$, so its benefit depends on how close h is to the true cost and on the map structure, not just on using a heuristic. Tie-breaking affects performance, and admissibility is what guarantees optimality.

9. Final Reflection

Why formulate the problem before the algorithm? The formulation fixes what a state is, which actions are legal, and what a solution means. A* is generic machinery over those definitions, so a wrong or vague

formulation gives code that runs but answers the wrong question. Writing it first also gives precise test cases (a no-solution map, a trivial map) and an unambiguous specification for the LLM.

In what sense is A* informed? Blind search such as BFS orders states only by how far they are from the start. A* also uses problem knowledge, the heuristic $h(n)$, an estimate of the remaining cost, so it orders states by estimated total cost $f = g + h$ and can favour states that seem nearer the goal.

Why does the heuristic matter? It determines both speed and correctness. An admissible heuristic preserves optimality, and the closer it is to the true cost the fewer states are expanded. Zero is safe but uninformative, and 2x Manhattan is faster but produced a 9-move path where 7 was optimal.

What did the LLM contribute? It translated the design into working code quickly (the heap-based frontier, stale-entry handling, path reconstruction, test scaffolding) and explained unfamiliar constructs. The design, the tests and the interpretation of results still needed the engineer.

What could go wrong with untested LLM code? Code can appear to work while hiding errors: non-optimal paths, infinite loops on unreachable goals, wrongly implemented heuristics, or misleading statistics. Here, untested output would also have given no hint that the heuristic had no effect on the lab map. The engineer remains responsible for validation.

Appendix A: Prompt Used with the LLM

The lab's suggested prompt, based on the design in Section 2. Replace or extend this with the exact prompts you actually used.

I am implementing a simple goal-based search agent in Python.
The environment is a grid represented by an ASCII map. The agent starts at S and must reach G. The symbols # represent obstacles and . represents free cells. The agent can move up, down, left, or right, and every movement has cost 1. Implement A* search. Use Manhattan distance as the heuristic:
$$h(n) = |x - x_G| + |y - y_G|.$$

The program should:

- represent grid positions as states;
- maintain an appropriate frontier;
- calculate $g(n)$, $h(n)$ and $f(n)$;
- avoid repeatedly expanding the same state;
- reconstruct the path when the goal is reached;
- report the path and its length;
- report the number of states expanded.

Keep the implementation simple and explain the main components of the code.

Follow-up prompt for Task 5: *Produce a BFS version of the same agent on the same map, reporting path, length and states expanded.*

Appendix C: Supplementary Experiment Code

Run from the same folder as search_agents.py.

```
import heapq, math
import search_agents as sa

def room(n, start_row, goal_row):
    rows = ["#" * (n + 2)]
    for r in range(n):
        line = ["#"] + "." * n + ["#"]
        if r == start_row: line[1] = "S"
        if r == goal_row: line[n] = "G"
        rows.append("".join(line))
    rows.append("#" * (n + 2))
    return rows

for label, grid in [("same row", room(15, 7, 7)), ("corners", room(15, 0, 14))]:
    b = sa.bfs(grid)
    a = sa.astar(grid, sa.manhattan)
    print(label, "BFS", b["states_expanded"], "A*", a["states_expanded"])

def astar_prefer_deeper(grid, heuristic):
    """A* whose ties are broken in favour of LARGER g."""
    start, goal = sa.parse_grid(grid)
    frontier = [(heuristic(start, goal), 0, 0, start)]
    g_cost = {start: 0}
    expanded = set()
    counter = 0
    while frontier:
        f, neg_g, _, current = heapq.heappop(frontier)
        g = -neg_g
        if g != g_cost.get(current, math.inf) or current in expanded:
            continue
        expanded.add(current)
        if current == goal:
            return g, len(expanded)
        for nb in sa.valid_neighbors(grid, current):
            t = g + 1
            if t < g_cost.get(nb, math.inf):
                g_cost[nb] = t
                counter += 1
                heapq.heappush(frontier, (t + heuristic(nb, goal), -t, counter, nb))

print(astar_prefer_deeper(room(15, 0, 14), sa.manhattan)) # (28, 29)

counterexample = ["#####", "#S...#", "#.##..#", "#.#...#", "#...G#", "#####"]
for name, h in sa.HEURISTICS.items():
    r = sa.astar(counterexample, h)
    print(name, r["path_length"], r["states_expanded"])
```

Appendix B: Full Program Listing (search_agents.py)

```
"""
Solved implementation for:
Artificial Intelligence Laboratory: Search and A*
Warehouse Robot Navigation

Includes:
- A* search with Manhattan / zero / Euclidean / scaled heuristics
- BFS comparison
- Path reconstruction
- State expansion counting
- Original, trivial, no-solution, and alternative-path tests
- Heuristic investigation
"""

from collections import deque
import heapq
import math

WAREHOUSE = [
    "#####",
    "#S...#.....#",
    "#.###.#####.",
    "#...#.....#.",
    "###.#####.#.",
    "#...#.....#.",
    "#.#####.#.",
    "#.....#G#",
    "#####",
]

TRIVIAL = [
    "####",
    "#SG##",
    "####",
]

NO_SOLUTION = [
    "#####",
    "#S...#",
    "###.###",
    "#...#G#",
    "#####",
]

ALTERNATIVE = [
    "#####",
    "#S...G#",
    "#....#",
    "#....#",
    "#####",
]

MOVES = [
    (-1, 0), # Up
    (1, 0), # Down
    (0, -1), # Left
    (0, 1), # Right
]

def parse_grid(grid):
    """Return start and goal coordinates."""
    start = None
    goal = None

    for r, row in enumerate(grid):
        for c, cell in enumerate(row):
            if cell == "S":
                start = (r, c)
            elif cell == "G":
                goal = (r, c)

    if start is None or goal is None:
        raise ValueError("Grid must contain both S and G.")

    return start, goal

def valid_neighbors(grid, state):
    """Generate valid states after Up/Down/Left/Right moves."""
    r, c = state

    for dr, dc in MOVES:
        nr, nc = r + dr, c + dc
```

```

        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                yield (nr, nc)

def reconstruct_path(came_from, start, goal):
    """Reconstruct path from goal back to start."""
    if goal not in came_from and goal != start:
        return None

    path = []
    current = goal

    while current != start:
        path.append(current)
        current = came_from[current]

    path.append(start)
    path.reverse()
    return path

def manhattan(state, goal):
    return abs(state[0] - goal[0]) + abs(state[1] - goal[1])

def euclidean(state, goal):
    return math.sqrt(
        (state[0] - goal[0]) ** 2 +
        (state[1] - goal[1]) ** 2
    )

def zero_heuristic(state, goal):
    return 0.0

def scaled_manhattan(state, goal):
    return 2.0 * manhattan(state, goal)

HEURISTICS = {
    "manhattan": manhattan,
    "zero": zero_heuristic,
    "euclidean": euclidean,
    "2x_manhattan": scaled_manhattan,
}

def astar(grid, heuristic=manhattan):
    """
    A* search.

    f(n) = g(n) + h(n)

    Returns a dictionary containing:
    - found
    - path
    - path_length
    - states_expanded
    """
    start, goal = parse_grid(grid)

    frontier = []
    counter = 0

    # Heap entries: (f, g, counter, state)
    heapq.heappush(
        frontier,
        (heuristic(start, goal), 0, counter, start)
    )

    came_from = {}
    g_cost = {start: 0}

    # Best-known expanded cost. A state may appear in the heap more than once
    # if a cheaper route to it is discovered later.
    expanded_g = {}

    states_expanded = 0

    while frontier:
        f, current_g, _, current = heapq.heappop(frontier)

        if current_g != g_cost.get(current, math.inf):
            continue

        if current in expanded_g and current_g >= expanded_g[current]:

```

```

        continue

    expanded_g[current] = current_g
    states_expanded += 1

    if current == goal:
        path = reconstruct_path(came_from, start, goal)
        return {
            "found": True,
            "path": path,
            "path_length": len(path) - 1,
            "states_expanded": states_expanded,
        }

    for neighbor in valid_neighbors(grid, current):
        tentative_g = current_g + 1

        if tentative_g < g_cost.get(neighbor, math.inf):
            g_cost[neighbor] = tentative_g
            came_from[neighbor] = current

            counter += 1
            h = heuristic(neighbor, goal)
            new_f = tentative_g + h

            heapq.heappush(
                frontier,
                (new_f, tentative_g, counter, neighbor)
            )

    return {
        "found": False,
        "path": None,
        "path_length": None,
        "states_expanded": states_expanded,
    }

def bfs(grid):
    """
    Breadth-first search for unit-cost movements.

    BFS is optimal here because every movement has cost 1.
    """
    start, goal = parse_grid(grid)

    frontier = deque([start])
    visited = {start}
    came_from = {}

    states_expanded = 0

    while frontier:
        current = frontier.popleft()
        states_expanded += 1

        if current == goal:
            path = reconstruct_path(came_from, start, goal)
            return {
                "found": True,
                "path": path,
                "path_length": len(path) - 1,
                "states_expanded": states_expanded,
            }

        for neighbor in valid_neighbors(grid, current):
            if neighbor not in visited:
                visited.add(neighbor)
                came_from[neighbor] = current
                frontier.append(neighbor)

    return {
        "found": False,
        "path": None,
        "path_length": None,
        "states_expanded": states_expanded,
    }

def mark_path(grid, path):
    """Return a printable grid with the solution path marked by *."""
    result = [list(row) for row in grid]

    if path:
        for r, c in path:
            if result[r][c] not in ("S", "G"):
                result[r][c] = "*"

    return "\n".join("".join(row) for row in result)

```

```

def print_result(name, result):
    print(f"\n{name}")
    print("." * 60)
    print("Solution found :", result["found"])
    print("Path length      :", result["path_length"])
    print("States expanded:", result["states_expanded"])

    if result["path"] is not None:
        print("Path:")
        print(result["path"])

def run_basic_tests():
    print("=" * 72)
    print("SEARCH LAB - SOLVED IMPLEMENTATION")
    print("=" * 72)

    print("\nORIGINAL WAREHOUSE")
    print("." * 60)
    print("\n".join(WAREHOUSE))

    result = astar(WAREHOUSE, manhattan)
    print_result("A* with Manhattan distance", result)

    if result["path"]:
        print("\nWarehouse with A* path:")
        print(mark_path(WAREHOUSE, result["path"]))

    print("\nTEST 2: TRIVIAL ONE-STEP CASE")
    trivial_result = astar(TRIVIAL, manhattan)
    print_result("Trivial A*", trivial_result)
    assert trivial_result["found"]
    assert trivial_result["path_length"] == 1

    print("\nTEST 3: NO-SOLUTION CASE")
    no_solution_result = astar(NO_SOLUTION, manhattan)
    print_result("No-solution A*", no_solution_result)
    assert not no_solution_result["found"]

    print("\nTEST 4: ALTERNATIVE PATHS")
    alternative_result = astar(ALTERNATIVE, manhattan)
    print_result("Alternative-path A*", alternative_result)
    assert alternative_result["found"]

    print("\nBasic tests passed.")

def compare_bfs_astar():
    print("\n" + "=" * 72)
    print("TASK 5: BFS VS A*")
    print("=" * 72)

    bfs_result = bfs(WAREHOUSE)
    astar_result = astar(WAREHOUSE, manhattan)

    print(f"\n{'Measure':<22} {'BFS':>15} {'A*':>15}")
    print("." * 54)
    print(
        f"{'Solution found':<22} "
        f"{'{str(bfs_result['found']):>15}'} "
        f"{'{str(astar_result['found']):>15}'}"
    )
    print(
        f"{'Path length':<22} "
        f"{'{str(bfs_result['path_length']):>15}'} "
        f"{'{str(astar_result['path_length']):>15}'}"
    )
    print(
        f"{'States expanded':<22} "
        f"{'{str(bfs_result['states_expanded']):>15}'} "
        f"{'{str(astar_result['states_expanded']):>15}'}"
    )

    # Since every action costs 1, BFS and A* with an admissible Manhattan
    # heuristic should return an optimal path.
    assert bfs_result["found"] == astar_result["found"]
    assert bfs_result["path_length"] == astar_result["path_length"]

def heuristic_investigation():
    print("\n" + "=" * 72)
    print("TASK 6: HEURISTIC INVESTIGATION")
    print("=" * 72)

    print(
        f"\n{'Heuristic':<20} {'Found':>10} "
        f"{'Path length':>15} {'States expanded':>18}"
    )

```

```

print("-" * 68)

for name, heuristic in HEURISTICS.items():
    result = astar(WAREHOUSE, heuristic)
    print(
        f"{name:<20} "
        f"{str(result['found']):>10} "
        f"{str(result['path_length']):>15} "
        f"{str(result['states_expanded']):>18}"
    )

def explain_design():
    print("\n" + "=" * 72)
    print("SEARCH-PROBLEM FORMULATION")
    print("=" * 72)

    print(
        """
State S:
    A grid position (row, column) of the robot.

Actions A:
    Up, Down, Left, Right.

Transition T:
    Applying a valid action moves the robot to the adjacent free cell.

Initial state s0:
    The cell containing S.

Goal G:
    The cell containing G.

Cost c:
    Every movement costs 1.

State information:
    The robot's current row and column are sufficient.

Invalid action:
    An action is invalid if it leaves the grid or enters an obstacle (#).

Determinism:
    Yes. A state and valid action determine exactly one next state.

Solution:
    A sequence of valid moves that takes the robot from S to G.
    """
    )

def main():
    explain_design()
    run_basic_tests()
    compare_bfs_astar()
    heuristic_investigation()

    print("\n" + "=" * 72)
    print("ALL EXPERIMENTS COMPLETED")
    print("=" * 72)
    print(
        "\nNote: A* with 2x Manhattan is intentionally included as an "
        "experiment. Unlike Manhattan distance, multiplying the heuristic "
        "by 2 can make it non-admissible, so optimality is not guaranteed."
    )

if __name__ == "__main__":
    main()

```